Quick Git Tutorial

https://gemini.google.com/app/ea7798b2b835a11f

User prompt: Please write me short tutorial how to use git

Response: Here is a short, practical tutorial to get you up and running with Git.

1. Installation & Setup

First, make sure Git is installed on your computer. You can check by running this in your terminal:

```
git --version
```

If it's installed, set your name and email (Git uses this to attach your identity to your work):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

2. Basic Workflow

Git tracks changes in a folder (called a **repository** or **repo**). Moving files into Git's history takes three main steps: **Stage**, **Commit**, and **Push**.

1 Initialize the Repository

Run this inside your project folder

Navigate to your project folder in the terminal and turn it into a Git repository:

```
git init
```

2 Check Status

See which files have changed

To see the current state of your files (what's modified, what's untracked):

```
git status
```

3 Stage Changes

Prepare files for saving

Tell Git which files you want to include in your next save (staging):

```
git add .
```

(The dot . adds all modified files in the current directory).

4 Commit Changes

Save your snapshot permanently

Save your staged changes to the local history with a descriptive message explaining what you did:

```
git commit -m "Add initial project files and homepage layout"
```

3. Working with Remote Repositories (GitHub/GitLab)

To back up your code online or collaborate with others, connect your local repository to a remote server (like GitHub).

1. **Connect to a remote repository:**
   `git remote add origin https://github.com/your-username/your-repo-name.git`
2. **Push your code online:** Send your committed changes to the main branch on GitHub:
   `git push -u origin main`

4. Essential Everyday Commands

Command	What it does
git clone <url>	Downloads an existing repository from the internet to your computer.

Command	What it does
git pull	Downloads the latest changes from the remote repository to your local machine.
git log	Displays the history of all commits made in the repository.
git branch	Lists all local branches in your repository.
git checkout -b <name>	Creates a new branch and switches to it immediately (useful for trying out new features safely).

User prompt: yes

Response: Branching and merging are the core features that make Git powerful for teamwork. They allow multiple people to work on different features at the same time without breaking the main codebase.

1. What is a Branch?

Think of a branch as an independent parallel timeline of your project. By default, your repository has one main timeline called main (or historically master).

When you want to build a new feature or fix a bug, you create your own branch so your experimental changes don't affect the working project.

- **Create and switch to a new branch:**
  `git checkout -b feature-login-page`

  (This creates a branch named feature-login-page and switches you to it immediately).
- **Switch back to the main branch:**
  `git checkout main`

2. How Merging Works

Once you finish your feature on your branch and commit your changes, you want to combine (merge) those changes back into the main branch.

1. Switch to the branch you want to merge *into* (usually main):
   `git checkout main`

2. Pull the latest updates from the remote repository to ensure your local main is up to date:
   `git pull`

3. Merge your feature branch into main:
   `git merge feature-login-page`

4. Push the updated main branch back online:
   `git push origin main`

3. Handling Merge Conflicts

A **merge conflict** happens when Git gets confused because two people (or you on two different branches) modified the *exact same line* of code in different ways, and Git doesn't know which version to keep.

When a conflict occurs, Git pauses the merge and marks the conflicting file. Here is how to fix it step-by-step:

1 Identify the Conflict

Git will pause and tell you which files have conflicts

Run git status to see which files are conflicting:

```
git status
```

2 Inspect Conflict Markers

Open the file in your code editor (like VS Code)

Git alters the file to show both versions using special symbols:

```
<<<<<<< HEAD
console.log("Welcome back, user!");
=======
console.log("Hello, welcome to our app!");
>>>>>>> feature-login-page
```

- Everything above `=======` is your current branch (main).
- Everything below `=======` is the incoming branch (feature-login-page).

3 Edit and Clean Up

Delete the markers and keep the code you actually want

Manually edit the file in your code editor to keep the desired code and remove the conflict markers

(<<<<<<<, =======, >>>>>>>). For example, keep just one line:

```
console.log("Welcome back, user!");
```

4 Stage the Resolution

Stage the fixed file

Tell Git the conflict has been resolved by staging the file:

```
git add <filename>
```

5 Complete the Commit

Finalize the merge

Finish the merge process by committing the resolution:

```
git commit -m "Resolve merge conflict in login script"
```
